

Prompting Strategy: CO-STAR Framework

Why is described by Zentennial in the article titled “Unlocking the Power of COSTAR Prompt Engineering”

- Context (C)
- Objective (O)
- Style (S)
- Tone (T)
- Audience (A)
- Response (R)

This strategy was chosen to ensure that GenAI receives all the necessary information to enhance its understanding of the problem, as well as have clear instructions regarding the objective and response.

Zentennial, F. (2024, January 19). Unlocking the Power of COSTAR Prompt Engineering: A Guide and Example on converting goals into.... *Medium*. <https://medium.com/@frugalzentennial/unlocking-the-power-of-costar-prompt-engineering-a-guide-and-example-on-converting-goals-into-dc5751ce9875>

Model	Task	Prompt Used	Output Summary	Decision
GPT4	Longest Shortest Path Function	Context : I am developing a computational model of Disinformation Dynamic spread using the Metropolis - Hastings Algorithm. Each agent are nodes in a network, each holding a discrete belief state: -1 (Opposed), 0 (Neutral), or 1 (Supportive) of the disinformation. I need to analyze how clusters of similar beliefs evolve structurally over time. Objective (O): Your task is to analyse the below code and create a Python function that calculates the longest shortest path for the subgraphs formed by agents of each specific belief state (-1, 0, and 1) and performed over the simulation's time steps. Style (S): Act as a research assistant proficient in complex system simulation and network science. Use technical, precise language, and ensure the code follows academic research standards – it is clean, modular, and commented. Tone (T): Analytical, helpful, and objective. Focus on algorithmic efficiency and clarity. Audience (A): The audience is a computational social science researcher. The explanation should be sophisticated enough to include in a research methodology section. Response (R): The output must include: 1. Modular python code compatible with common libraries. 2. An explanation of the chosen algorithm and why it fits this specific belief-state filtering. 3. Performance analysis (time complexity and potential bottlenecks) 4. Clear explanation of each block.	The description of the problem and code.	Incorporated the proposed code but created two tests to check the functionality. One on a 4 negative nodes which are connected into a line graph, the output was correctly 3. The second test checked if the the same 4 negative nodes but which were only connected: (0-1) and (2-3). These were also correctly analysed by the code to have the longest diameter of 1. Additionally, we have analysed the code to see if it can be sped up.
GPT4	Calculating the self-consistency equation for magnetization	Write python code for self-consistency equation for magnetization	Code for implementing the equation and explanation of the functionality.	Decided not to use it as is. The paper also only conceptually explains it, but never actually implements it.
GPT4	Speeding up the longest diameter functions	Change the connected_components_by_belief to incorporate vectorized numpy bfs - a boolean mask for visited and allowed. Or propose an alternative way to improve efficiency.	(i) what can realistically be vectorized, (ii) a high-performance NumPy-based BFS variant, and (iii) a	Not everything can be vectorised, for example the BFS loop cannot be, however, the visited/allowed can be turned into boolean masks. Since our network is static – we do not add

			superior alternative approach for your use case.	additional nodes into the network after initial creation – it is more beneficial to be extracting the neighbours arrays once and then reuse them throughout the code. based on the new precompute_neighbours and the new connected components precomputed I adapted the previous code for the bfs_distances, component_state and belief_state_diameters_over_time. Previously this function also took the adjacency matrix and extracted the neighbours within itself.
GPT5	Scale Free Network Adjacency generation	Context : I am developing a computational model of Disinformation Dynamic spread using the Metropolis - Hastings Algorithm. Each agent are nodes in a network, each holding a discrete belief state: -1 (Opposed), 0 (Neutral), or 1 (Supportive) of the disinformation. I need to analyze how clusters of similar beliefs evolve structurally over time. Objective (O): Your task is to assist in developing a function to create a scale free adjacency matrix. Below you will be provided with a concept for the code, based on it create an optimised function using jax in Python. Make comprehensive comments in the code. Style (S): Act as a research assistant proficient in complex system simulation and network science. Use technical, precise language, and ensure the code follows academic research standards – it is clean, modular, and commented. Tone (T): Analytical, helpful, and objective. Focus on algorithmic efficiency and clarity. Audience (A): The audience is a computational social science researcher. The explanation should be sophisticated enough to include in a research methodology section. Response (R): The output must include: 5. Modular python code compatible with common libraries. 6. An explanation of the chosen algorithm and why it fits this specific belief-state filtering. 7. Performance analysis (time complexity and potential bottlenecks) 8. Clear explanation of each block. import numpy as np import matplotlib.pyplot as plt <pre>def generate_scale_free_sparse_adjacency_matrix_jax(num_nodes: int, num_edge: int, weight_range: (-1.0, 1.0), seed: int = 0,): """ Generate a scale-free network using the Barabási–Albert preferential attachment model, implemented in JAX. num_nodes : int Total number of nodes. num_edge : int Number of edges each new node attaches to existing nodes. weight_range : (float, float) Range for symmetric edge weights. seed : int</pre>	1. Concept overview 2. Code with JAX 3. Through explanation 4. Performance analysis - $O(N^2)$	The code required some manual debugging. However, overall logic was functional and follows the pseudo code given to the LLM. The extensive prompt helped achieve the intended outcome.

		<pre> Random seed for reproducibility. :rtype: sparse_adj_mat.Sparse_Adjacency_Matrix """ #Keys key = jax.random.PRNGKey(seed) w_min, w_max = weight_range # initial complete graph size init_size = m + 1 # initialize a small network #function to sequentially attach the nodes def attach_node(state, node_id): #the probability the new node connects to existing node i is proportional to i's degree: probs = degrees[:node_id] / jnp.sum(degrees[:node_id]) # update degrees return (adj, degrees, key) #at each timestep add a new node with m<= m0 edges (adj, _, _) = jax.lax.scan(attach_node, (adj, degrees, key), jnp.arange(init_size, num_nodes),) #return the sparse adjacency of the adjacency return Sparse_Adjacency_Matrix(adj) </pre>		
GPT5	Small world adjacency matrix generation	now create a code for small world adjacency matrix based on the pseudo code attached below. Use jax in python <pre> def generate_small_world_sparse_adjacency_matrix_jax(num_nodes: int, k: int, p: float, weight_range: (-1.0, 1.0), seed: int = 0,): """ Generate a small-world network using the Watts–Strogatz model. Start with a regular lattice, where n nodes are connected to k neighbors, then the algorithm rewires edges to random nodes with probability p. Parameters ----- num_nodes : int Number of nodes in the network. </pre>	Algorithmic rationales JAX code Explanation of each block And performance analysis The code follows the pseudo code prompted.	BA is sequential, thus full vectorised BA is not possible. However, jax can be used for sequential node addition. This code was tested and added to the pipeline. Also added a part with assigning random symmetrical weights to existing edges as it was not created by the LLM.

		<pre> k : int Number of nearest neighbours of each node. p: Rewiring probability. weight_range : (float, float) Range of symmetric edge weights. seed : int Seed for reproducibility. Returns ----- Sparse_Adjacency_Matrix Weighted symmetric sparse adjacency matrix. """ #paramter control """ If p=0 then the network remains a fully ordered regular lattice If p=1 - network is completely random . if 0<p<1 - small world. """ #1. iitialize ring lattice - nodes are arranged in a circle, with each node connected to its k nearest neighbors adj = jnp.zeros((num_nodes, num_nodes)) #function to create the circle def connect_ring(i, adj): # rewiring function def rewire_edge: #reiwre the edges (adj, key), _ = jax.lax.scan(rewire_edge) #do not add weights to the edges return sparse_adj_mat.Sparse_Adjacency_Matrix(adj) </pre>		
	VS Code copilot – correct abug in the small world function	debug	The problem is that <code>jax.random.choice</code> needs a concrete array size. The issue is fundamental with trying to use JAX's JIT with dynamic values. I need to use a fixed-size array approach. Let me rewrite the function	Code was corrected

			to use a fixed-size probability array:	
	VS Code copilot – correct a bug in the scale free function	AssertionError: The passed adjacency matrix should be symmetric but it seems not to be.	The problem is on the last line before the return. After applying the weights, the matrix is no longer symmetric because random weights are applied element-wise. We need to make it symmetric by averaging. <pre># Make symmetric by averaging weighted_adj = 0.5 * (weighted_adj + weighted_adj.T)</pre>	Code was corrected
GPT5	Gif generation	Act as a research assistant proficient in complex system simulation and network science. Use technical, precise language, and ensure the code follows academic research standards – it is clean, modular, and commented. There are separate pickle files in a folder. Each file contains arrays with 10 different runs of a network and its name is erdos_renyi_{lambda_value}. Write code to read this data and output a gif where each frame is a run for a different value of lambda. The plot should follow the structure: <pre>plt.xlabel("Time steps") plt.ylabel("Magnetization") plt.ylim(-1,1) plt.title(f"Impact of {topology_name} topology on belief alignment \n {param_name} point: $\beta = \{\beta\}$, $\lambda = \{\lambda\}$") plt.grid(True, alpha=0.3) plt.legend() plt.show()</pre> Or suggest a different approach for a dynamic picture to output this data	Code for a gif animation with mean trajectory and std error bars.	The code was created for mean trajectory, vs I wanted a code that would plot all the individual trajectories. Therefore, I adapted the code to only output the individual trajectories. LLM did not fulfill the task, possibly due to the vagueness of the prompt.

AI use Theo:

Using GPT-4o, 23.01.2026 11:55:31

Context: This is for when I wanted to use JAX to speed up the simulation, and I foresaw some conflict between the dynamic sizes of amounts of neighbors and the fixed size requirements for JAX.

Prompt:

Hello! I have this project where I want to speed up my code. It's a kind of Ising model simulation on arbitrary graphs. I want to be able to efficiently get neighbors and their edge weights. So far I am doing it with numpy in the "sparse_adjacency_matrix" class.

In my step() function in the BeliefNetwork, I am looping over all nodes in a random order. However, I could just do them all in parallel / vectorized which is what I am hoping to do. I would like to use JAX for this so that it can be compiled and fast. However, I am wondering how and if I'd be able to use my existing sparse_adjacency_matrix function to get neighbors, because it returns arrays of dynamic sizes.

What could I do to make it vmap / pmappable? Or are there other obvious ways to speed up my code?

Reply (paraphrased / summarized):

Suggestions about using a dense matrix (I don't want this)

Suggestion about precomputing fixed-size neighborhood arrays (I think this is what we need)

Prompt:

Precomputing the fixed-size neighbor arrays seems sensible. But, could I maybe just use some python multiprocessing to speed up the for loop too? Or would JAX always beat this?

Reply (paraphrased / summarized):

JAX will be faster but a bit more cumbersome to implement. (So I will do this!)

Prompt:

Alright. Let's then precompute the neighborhood information in a fixed size array. For this we will need to know the maximum degree as well, which can be inferred.

Please add a function to `sparse_adjacency_matrix` that spits out these two contiguous 2D arrays, one with neighborhoods idxs (maybe with -1 for None) and one with weights (with 0 for None)

Reply:

Some code that I used in the `Sparse_Adjacency_Matrix` class in the `precompute` function